



Systeme de Réservations de Salles

Implémentations des Designs Patterns

Michel MUSTAFOV

Edwin WEHBE





TABLE DES CONTENUS

1 Introduction

2 Architecture du Projet

3 Vue d'ensemble des Design Patterns

4 Creational Patterns

4.1 Singleton Pattern

4.2 Factory Method Pattern

5 Structural Patterns

5.1 Composite Pattern

5.2 Flyweight Pattern

5.3 Adapter Pattern

6 Behavioral Patterns

6.1 Strategy Pattern

6.2 Observer Pattern

7 Diagramme de classes UML

8 Exécution et tests

8.1 Prérequis

8.2 Démarrage rapide

8.3 Exécution des tests

8.4 Endpoints API



1. Introduction

Ce document décrit l'implémentation d'un Room Reservation System développé en Rust. Le projet démontre l'application pratique de 7 design patterns issus du catalogue Gang of Four (GoF).

L'objectif est de montrer comment les design patterns résolvent des problèmes architecturaux réels, notamment : garantir l'unicité des instances, créer des objets de manière flexible, gérer des hiérarchies, optimiser l'utilisation de la mémoire, adapter des interfaces, changer des algorithmes dynamiquement, découpler la gestion des événements.

2. Architecture du projet

Le projet suit une architecture modulaire:

Couche	Emplacement	Responsabilité
Models	src/models/	Structures de données : Room, Reservation
Patterns	src/patterns/	Implémentation des 7 design patterns
Services	src/services/	Logique métier, gestion de l'état
API	src/main.rs	Serveur HTTP, endpoints REST
Frontend	static/	Interface web : HTML, CSS, JavaScript

Structure des fichiers:

src/patterns/

— singleton.rs	→ ActivityLogger, ConfigManager
— factory.rs	→ RoomFactory trait + concrete factories
— composite.rs	→ Reservable trait, Room, RoomGroup
— flyweight.rs	→ RoomTypeInfo, RoomTypeFlyweight
— strategy.rs	→ ValidationStrategy + implementations
— observer.rs	→ ReservationObserver, EventPublisher
— adapter.rs	→ CalendarAdapter + implementations

3. Vue d'ensemble des Design Patterns

Résumé des 7 design patterns implémentés :

Pattern	Catégorie	Problème résolu
Singleton	Creational	Garantit UN seul logger et une seule configuration globale
Factory Method	Creational	Crée des salles par type sans exposer la logique d'instanciation
Composite	Structural	Traite uniformément les salles individuelles et les groupes
Flyweight	Structural	Partage les métadonnées des types de salles pour économiser la mémoire
Adapter	Structural	Intègre des APIs de calendriers externes avec une interface unifiée
Strategy	Behavioral	Change les règles de validation à l'exécution selon le rôle
Observer	Behavioral	Notifie plusieurs systèmes lors des événements de réservation



4. Creational Patterns

4.1 Singleton Pattern

Problème	Il est nécessaire d'avoir exactement UNE seule instance de ActivityLogger et ConfigManager dans toute l'application.
Solution	Utilisation de once_cell::Lazy avec Arc<RwLock<T>> pour une initialisation paresseuse thread-safe.
Emplacement	src/patterns/singleton.rs
Composants clés	ActivityLogger, ConfigManager, ACTIVITY_LOG (global static), CONFIG (global static)

Fonctionnements:

- ACTIVITY_LOG est initialisé lors du premier accès puis réutilisé partout
- Thread-safe : plusieurs requêtes HTTP peuvent écrire simultanément grâce à RwLock
- Stocke les 500 dernières activités (réservations, annulations, erreurs)

4.2 Factory Method Pattern

Problème	Créer différents types de salles (Conference, Meeting, Training) avec des équipements par défaut spécifiques.
Solution	Définition du trait RoomFactory. Chaque factory concrète crée des salles avec les paramètres appropriés.
Emplacement	src/patterns/factory.rs
Composants clés	RoomFactory (trait), ConferenceRoomFactory, MeetingRoomFactory, TrainingRoomFactory, RoomFactoryManager

Types de factories et équipements par défaut:

Factory	Équipement par défaut
ConferenceRoomFactory	Projector, Video Conference, Sound System
MeetingRoomFactory	Whiteboard, Phone, WiFi
TrainingRoomFactory	Projector, Whiteboard, Computers
AuditoriumFactory	Microphone, Sound System, Stage Lighting



5. Structural Patterns

5.1 Composite Pattern

Problème	Pouvoir réserver un étage ou un département aussi facilement qu'une salle individuelle.
Solution	Le trait Reservable est implémenté par Room (leaf) et RoomGroup (composite).
Emplacement	src/patterns/composite.rs
Composants clés	Reservable (trait), Room, RoomGroup, RoomGroupType (Floor/Building/Department)

Exemple : Appeler `total_capacity()` sur un Floor additionne récursivement la capacité de toutes les salles.

5.2 Flyweight Pattern

Problème	100 salles de type Conference stockant la même description consomment inutilement de la mémoire.
Solution	Partage des informations immuables via des pointeurs Arc. Toutes les salles Conference pointent vers UNE seule description.
Emplacement	src/patterns/flyweight.rs
Composants clés	RoomTypeInfo (données partagées), RoomTypeFlyweight (gestion du cache), FlyweightRoom

5.3 Adapter Pattern

Problème	Google Calendar, Outlook et iCal ont des APIs différentes, nécessitant sinon de multiples conditions if/else.
Solution	Le trait CalendarAdapter fournit une interface unifiée. Chaque adapter traduit vers une API spécifique.
Emplacement	src/patterns/adapter.rs
Composants clés	CalendarAdapter (trait), GoogleCalendarAdapter, OutlookAdapter, ICalAdapter

Méthodes unifiées:

- `sync_event(reservation)` → Creates event in external calendar
- `delete_event(id)` → Removes event from external calendar
- `check_conflicts(time)` → Checks availability in external calendar



6. Behavioral Patterns

6.1 Strategy Pattern

Problème	Les règles de validation diffèrent selon le type d'utilisateur : Standard (4h max), VIP (8h), Admin (illimité)
Solution	Trait ValidationStrategy avec des implémentations interchangeables, sélectionnées à l'exécution.
Emplacement	src/patterns/strategy.rs
Composants clés	ValidationStrategy (trait), StandardValidation, VipValidation, AdminValidation

Règles de validation:

Strategy	Durée max	Réservation à l'avance	Week-end
Standard	4 heures	30 jours	Non
VIP	8 heures	90 jours	Oui
Admin	Illimité	Illimité	Oui

6.2 Observer Pattern

Problème	Lorsqu'une réservation est créée, il faut notifier plusieurs systèmes sans couplage fort.
Solution	EventPublisher notifie tous les observers abonnés. Chaque observer réagit indépendamment.
Emplacement	src/patterns/observer.rs
Composants clés	ReservationObserver (trait), EventPublisher, EmailNotifier, AuditLogger, SlackNotifier

Types d'événements:

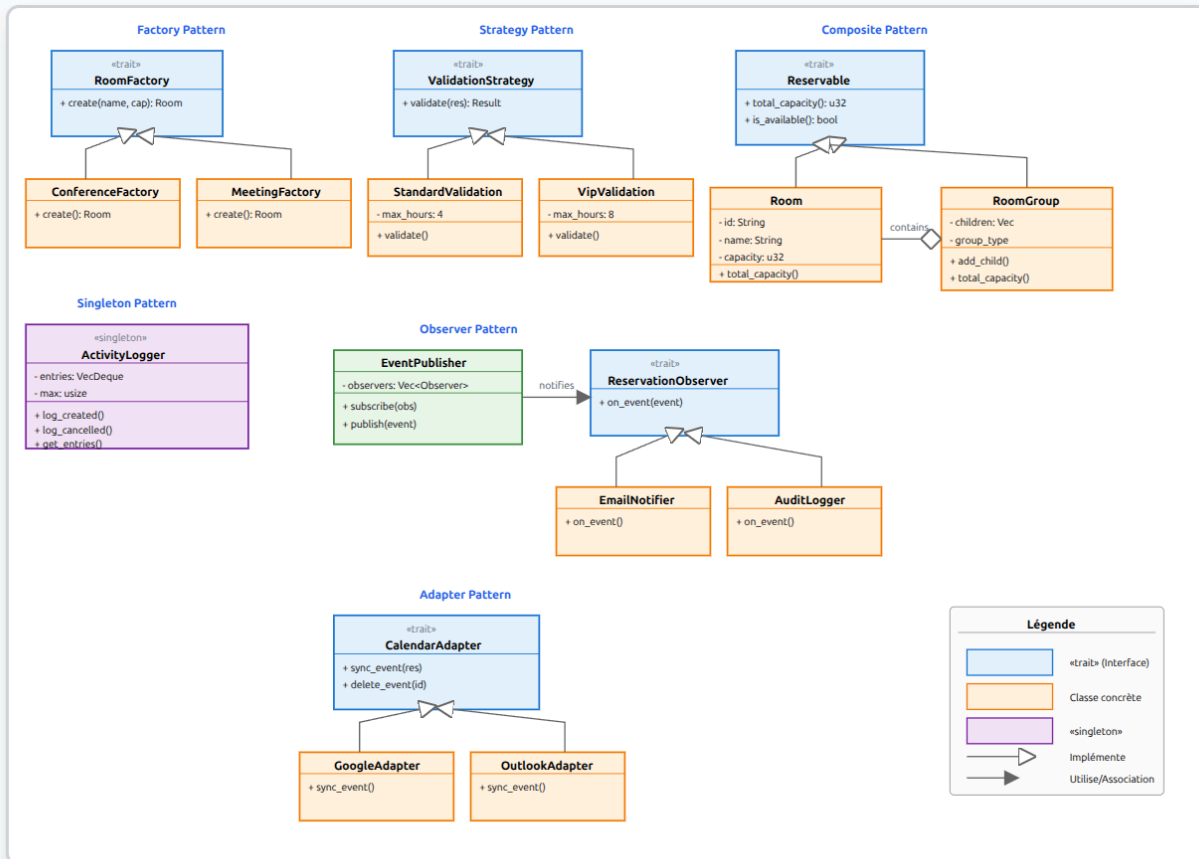
- ReservationCreated → Confirmation mail + Audit log + Analytics tracking
- ReservationCancelled → Annulation mail + Audit log + Notification Slack
- ReservationConfirmed → Rappel email + Calendar synchronisation calendrier



7. Diagramme de classes UML

Le diagramme UML illustre les relations entre les principales classes et design patterns:

Système de réservation de salles - Diagramme de classes UML



Patrons de conception implémentés

Patron Fabrique (Factory) : RoomFactory crée différents types de salles

Patron Stratégie (Strategy) : ValidationStrategy fournit différentes règles de validation

Patron Composite : Room et RoomGroup implémentent Reservable

Patron Singleton : ActivityLogger garantit une instance unique

Patron Observateur (Observer) : EventPublisher notifie les ReservationObservers

Patron Adaptateur (Adapter) : CalendarAdapter intègre des systèmes de calendrier externes



8. Exécution et tests

8.1 Prérequis

- Rust 1.75+ → Installer sur <https://rustup.rs>

8.2 Démarrage rapide

```
# 1. Extraire le projet
unzip room-reservation-system.zip
cd room-reservation-system

# 2. Build et run
cargo run --release

# 3. Ouvrir http://127.0.0.1:8080
```

8.3 Exécution des tests

```
cargo test
# Résultat : 23 tests réussis
```

8.4 Endpoints API

Méthode	Endpoint	Description
GET	/api/rooms	Liste toutes les salles
POST	/api/rooms	Crée une salle (Factory Pattern)
GET	/api/reservations	Liste les réservations
POST	/api/reservations	Crée une réservation (Strategy Pattern)
DELETE	/api/reservations/{id}	Annule une réservation (Observer Pattern)
GET	/api/logs	Consulte le journal d'activité (Singleton)